Shipping Networks as Directed Containers

Extracted from a machine-checked development

Abstract

Every directed (multi)graph G determines a directed container whose shapes are the nodes of G and whose positions at a node v are the finite directed paths starting at v, recorded together with their endpoints. The root is the empty path, the sub-shape map returns the endpoint of a path, and the shift is path concatenation. As a category this construction is the free category on G. We record the elementary path algebra, the directed-container laws, and a concrete running example—an eleven-facility wine shipping network—with explicit route counts. The proofs of all stated results have been machine-checked; we record only the mathematical content.

Contents

1 Introduction **1**

2 Directed graphs and paths **2**

3 The network directed container **3**

4 Running example: the Sawasdee Cellars wine network **3**

5 Discussion **4**

1 Introduction

In the theory of directed containers, a structure of shapes, positions, roots, and shifts axiomatises a free category on a set of generators. A natural source of examples is directed graphs: nodes generate shapes, and directed paths generate positions. The present note isolates that construction.

Given a directed multigraph G (allowing parallel edges and loops, with no finiteness assumption), one forms a directed container network(G) by taking shapes to be nodes and positions at v to be pairs $\langle w, \rho \rangle$ consisting of an endpoint w and a finite directed path ρ from v to w. Concatenation of paths supplies the monoidal structure on positions required by the directed-container axioms. Viewed as a category (via the free category associated to a directed container), network(G) is the free category on G.

The development works throughout in a single universe of types. The statements below are those of a formal development in dependent type theory; proofs are indicated by the phrase "by lean," meaning they have been checked by a proof assistant rather than written out in prose.

What this note claims. In ordinary mathematical language: the free category on a directed graph admits a presentation as a directed container whose positions are path-endpoint pairs. The directed-container laws reduce almost definitionally to the unit and associativity laws for

path concatenation. On a concrete acyclic shipping graph with eleven facilities, the construction is inhabited and yields the expected route cardinalities (nine routes out of the first vineyard; a single trivial route at each retail sink).

2 Directed graphs and paths

Definition 2.1 (Directed graph)**.** A **directed (multi)graph** G consists of

1. a type V of *nodes* (vertices),
2. a type E of *edges*, and
3. endpoint maps $\mathrm{src} : E \to V$ and $\mathrm{tgt} : E \to V$.

Parallel edges and loops are allowed. No finiteness of V or E is assumed.

Definition 2.2 (Directed path)**.** Let G be a directed graph. For nodes $v, w \in V$, the type $\mathrm{Path}_G(v, w)$ of **finite directed paths** from v to w is defined inductively by

1. $\mathrm{nil}(v) : \mathrm{Path}_G(v, v)$ (the empty path at v), and
2. if $e \in E$ and $p : \mathrm{Path}_G\big(\mathrm{tgt}(e),\, w\big)$, then

$$\mathrm{cons}(e, p) \;:\; \mathrm{Path}_G\big(\mathrm{src}(e),\, w\big)$$

   (follow e, then continue along p).

Thus paths are "cons-at-the-front": an outgoing edge followed by a path from its target.

Definition 2.3 (Path concatenation)**.** Given paths $p : \mathrm{Path}_G(v, w)$ and $q : \mathrm{Path}_G(w, u)$, their **concatenation** $p +\!\!+ q : \mathrm{Path}_G(v, u)$ is defined by recursion on p:

$$\mathrm{nil}(v) +\!\!+ q \;=\; q, \tag{1}$$
$$\mathrm{cons}(e, p) +\!\!+ q \;=\; \mathrm{cons}\big(e,\, p +\!\!+ q\big). \tag{2}$$

Proposition 2.4 (Left unit)**.** *For every path* $p : \mathrm{Path}_G(v, w)$*,*

$$\mathrm{nil}(v) +\!\!+ p \;=\; p.$$

Proof. By lean. □

Proposition 2.5 (Right unit)**.** *For every path* $p : \mathrm{Path}_G(v, w)$*,*

$$p +\!\!+ \mathrm{nil}(w) \;=\; p.$$

Proof. By lean. □

Proposition 2.6 (Associativity)**.** *For paths* $p : \mathrm{Path}_G(v, w)$*,* $q : \mathrm{Path}_G(w, u)$*, and* $r : \mathrm{Path}_G(u, x)$*,*

$$(p +\!\!+ q) +\!\!+ r \;=\; p +\!\!+ (q +\!\!+ r).$$

Proof. By lean. □

The left unit law holds definitionally. The right unit and associativity laws are proved by induction on the first path.

3 The network directed container

We recall the data of a directed container, specialised to the form used below. A directed container consists of a type of shapes, a family of position types, a distinguished root position at each shape, a sub-shape map sending a shape and a position to a shape, and a shift operation composing positions, subject to five equational laws (identity and associativity for shift, together with compatibility of sub with root and shift).

Definition 3.1 (Network directed container)**.** Let G be a directed graph. The **network directed container** network(G) is defined by

$$\mathrm{Shape}\big(\mathrm{network}(G)\big) \;=\; V, \tag{3}$$

$$\mathrm{Pos}\big(\mathrm{network}(G)\big)(v) \;=\; \sum_{w\,:\,V} \mathrm{Path}_G(v,w), \tag{4}$$

$$\mathrm{root}(v) \;=\; \langle v,\, \mathrm{nil}(v)\rangle, \tag{5}$$

$$\mathrm{sub}\big(v,\, \langle w,\rho\rangle\big) \;=\; w, \tag{6}$$

$$\mathrm{shift}\big(v,\, \langle w,\rho\rangle,\, \langle u,\sigma\rangle\big) \;=\; \langle u,\, \rho \mathbin{+\!\!+} \sigma\rangle. \tag{7}$$

Here a position at v is a dependent pair $\langle w,\rho\rangle$ of an endpoint and a path from v to that endpoint; the sub-shape is the first projection; the root is the empty path; and shift is path concatenation.

Remark 3.2 (Directed-container laws)**.** With this packaging, several of the directed-container axioms hold by definitional equality: the endpoint of the empty path is the node itself, and the endpoint of a concatenation is the endpoint of the second path. The remaining laws are precisely the right unit law for concatenation (Proposition 2.5) and associativity (Proposition 2.6), each transported along the dependent pair of endpoint and path. Indexing paths by both endpoints—rather than by start alone—is what makes the sub-shape a pure first projection and collapses the transport-heavy equalities to the path algebra above.

Proposition 3.3 (Network is a directed container)**.** *For every directed graph G, the data of Definition 3.1 satisfy the directed-container axioms.*

Proof. By lean. □

Remark 3.4. Finiteness of G is not assumed. Cyclic graphs yield infinitely many positions at a node; that phenomenon is treated separately in the graph-rung analysis of the associated development.

4 Running example: the Sawasdee Cellars wine network

We instantiate the construction on a concrete shipping graph: three South Australian vineyards ship through an export agency and a Melbourne–Chonburi sea-freight leg to two Thai distributors and three retail shops. The graph is a polytree (thinness is treated elsewhere); here we only form the network and count routes.

Definition 4.1 (Sawasdee facilities and legs)**.** Let Facility be the eleven-element type

$$\begin{aligned}&\mathrm{vineyard}_1,\ \mathrm{vineyard}_2,\ \mathrm{vineyard}_3,\ \mathrm{sawasdee},\ \mathrm{melbournePort},\ \mathrm{thaiPort},\\ &\mathrm{distributor}_1,\ \mathrm{distributor}_2,\ \mathrm{retail}_1,\ \mathrm{retail}_2,\ \mathrm{retail}_3,\end{aligned}$$

and let Leg be the ten-element type of shipping legs

$v_1s,\ v_2s,\ v_3s$	(vineyards $\to$ Sawasdee),
smp	(Sawasdee $\to$ Melbourne Port),
sea	(Melbourne Port $\to$ Thai Port),
$td_1,\ td_2$	(Thai Port $\to$ distributors),
$d_1r_1,\ d_1r_2$	(Distributor$_1$ $\to$ Retail$_1$, Retail$_2$),
d_2r_3	(Distributor$_2$ $\to$ Retail$_3$).

Write G_{wine} for the directed graph with these nodes and edges and the evident source and target maps, and write $\mathcal{N}_{\text{wine}} := \text{network}(G_{\text{wine}})$ for the associated directed container.

A **route** out of a facility v is a position of $\mathcal{N}_{\text{wine}}$ at v, i.e. a pair $\langle w, \rho\rangle$ with $\rho : \text{Path}_{G_{\text{wine}}}(v, w)$. Given a leg e and a route out of $\text{tgt}(e)$, one obtains a route out of $\text{src}(e)$ by consing e onto the underlying path.

Proposition 4.2 (Route counts)**.** *In* G_{wine}*:*

1. *There are exactly nine routes out of* vineyard$_1$ *(the empty route, together with one route to each of the eight facilities reachable from* vineyard$_1$*).*
2. *There is exactly one route out of* retail$_1$ *(the empty route: a retail shop is a sink).*
3. *The endpoints of the nine routes out of* vineyard$_1$ *are, in order of enumeration,*

$$\text{vineyard}_1,\ \text{sawasdee},\ \text{melbournePort},\ \text{thaiPort},$$
$$\text{distributor}_1,\ \text{retail}_1,\ \text{retail}_2,\ \text{distributor}_2,\ \text{retail}_3.$$

Proof. By lean. □

Remark 4.3. The enumerations are constructed in reverse topological order, so that each facility's route list is assembled from the lists of facilities strictly downstream. Completeness of the lists for this acyclic graph is immediate by inspection; the formal checks assert only the stated lengths and endpoints.

Proposition 4.4 (Root and sub on a concrete route)**.** *Let* ρ *be the full five-leg route* vineyard$_1$ $\to$ sawasdee $\to$ melbournePort $\to$ thaiPort $\to$ distributor$_1$ $\to$ retail$_1$*. Then:*

1. *the empty route at* vineyard$_1$ *coincides with the root position of* $\mathcal{N}_{\text{wine}}$ *at* vineyard$_1$*;*
2. $\text{sub}(\text{vineyard}_1, \rho) = \text{retail}_1$*;*
3. *shifting the root by* ρ *recovers* ρ*.*

Proof. By lean. □

Thus on the running example, sub is literally "where the route ends," and shift specialises to path concatenation of an empty prefix with a concrete route.

5 Discussion

What has been established. Every directed graph presents a directed container whose positions are path-endpoint pairs and whose monoidal structure is concatenation. The construction is free on the graph: as a category it is the free category generated by the edges. The packaging with double-indexed paths makes most of the directed-container equalities definitional, leaving only the classical unit and associativity laws of path composition.

Scope. The development does not assume finiteness, does not prove thinness of polytree networks (treated separately), and does not address trackings, aggregations, or other structure on top of network(G). Those refinements build on the same foundations.

Machine checking. Every definition and theorem statement above corresponds to a declaration in a machine-checked formalisation. The phrase "by lean" indicates that the proof has been verified by a proof assistant; we have not reproduced the formal proof scripts here.

Acknowledgements

The mathematical content follows the construction of shipping networks as directed containers in the supply-chain development that motivated this note. The presentation is a prose rendering of a formal development intended for human review of the theorem statements.

References

[1] M. Abbott, T. Altenkirch, and N. Ghani. Containers: constructing strictly positive types. *Theoretical Computer Science*, 342(1):3–27, 2005.

[2] D. Ahman and T. Uustalu. Coalgebraic update lenses. In *MFPS*, volume 308 of *ENTCS*, pages 25–48, 2014.